

Spring Boot Data Management Application

1. Approach

The objective of this project is to build a Spring Boot MVC application to manage two related entities: `Author` and `Book`. We chose a **One-to-Many relationship**, where one `Author` can write multiple `Books`, but a `Book` is associated with only one `Author`.

The project follows a standard layered architecture:

- **Model (Entity Layer):** Contains `Author` and `Book` entities mapped to the database using JPA annotations.
- **Repository (Data Access Layer):** Extends `JpaRepository` for basic CRUD and includes a custom `@Query` for fetching `Book` details alongside `Author` data.
- **Service Layer:** Connects the `Controller` and the `Repository`, handling business logic like finding and updating records.
- **Controller Layer:** A standard MVC `@Controller` handles HTTP GET and POST requests, managing data flow between models and JSP pages.
- **View Layer:** JavaServer Pages (JSP) coupled with JSTL rendering the User Interface.

2. Entity Relationship Design (ER Design)

- **Author Entity:**
 - `id` (Primary Key, Auto-increment)
 - `name` (String, Non-null)
 - `nationality` (String)
 - `books` (List of `Book`, mapped by `author`)
- **Book Entity:**
 - `id` (Primary Key, Auto-increment)
 - `title` (String, Non-null)
 - `publicationYear` (Integer)
 - `author_id` (Foreign Key linked to `Author ID`)

Both entities are related via `@OneToMany` and `@ManyToOne` annotations, ensuring foreign key constraints in the underlying H2 embedded database.

3. Implementation Details for Operations

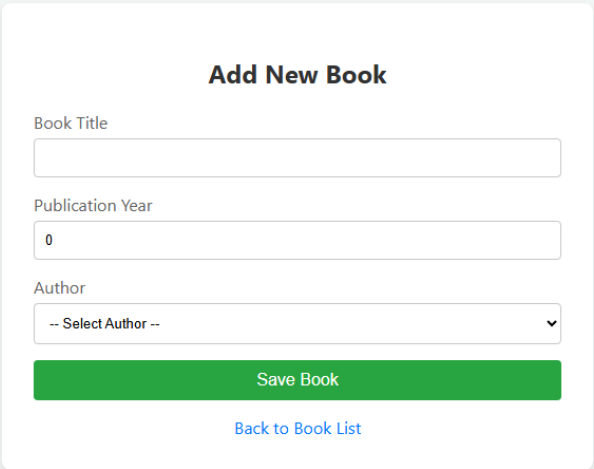
Populate Database

The database schema is automatically generated using `spring.jpa.hibernate.ddl-auto=update` . We populated it using a `data.sql` file that executes automatically after DDL scripts run. We included 10 row inserts for `Author` and 10 for `Book` .

Create Operation

- **Code implementation:** Handled in `BookController.java` inside the `showAddForm()` and `addBook()` methods.
- UI forms bind to a new `Book` Model object.
- Exception handling intercepts `DataIntegrityViolationException` to warn if missing fields/associations exist.

Screenshot of Create Operation:



Add New Book

Book Title

Publication Year

Author

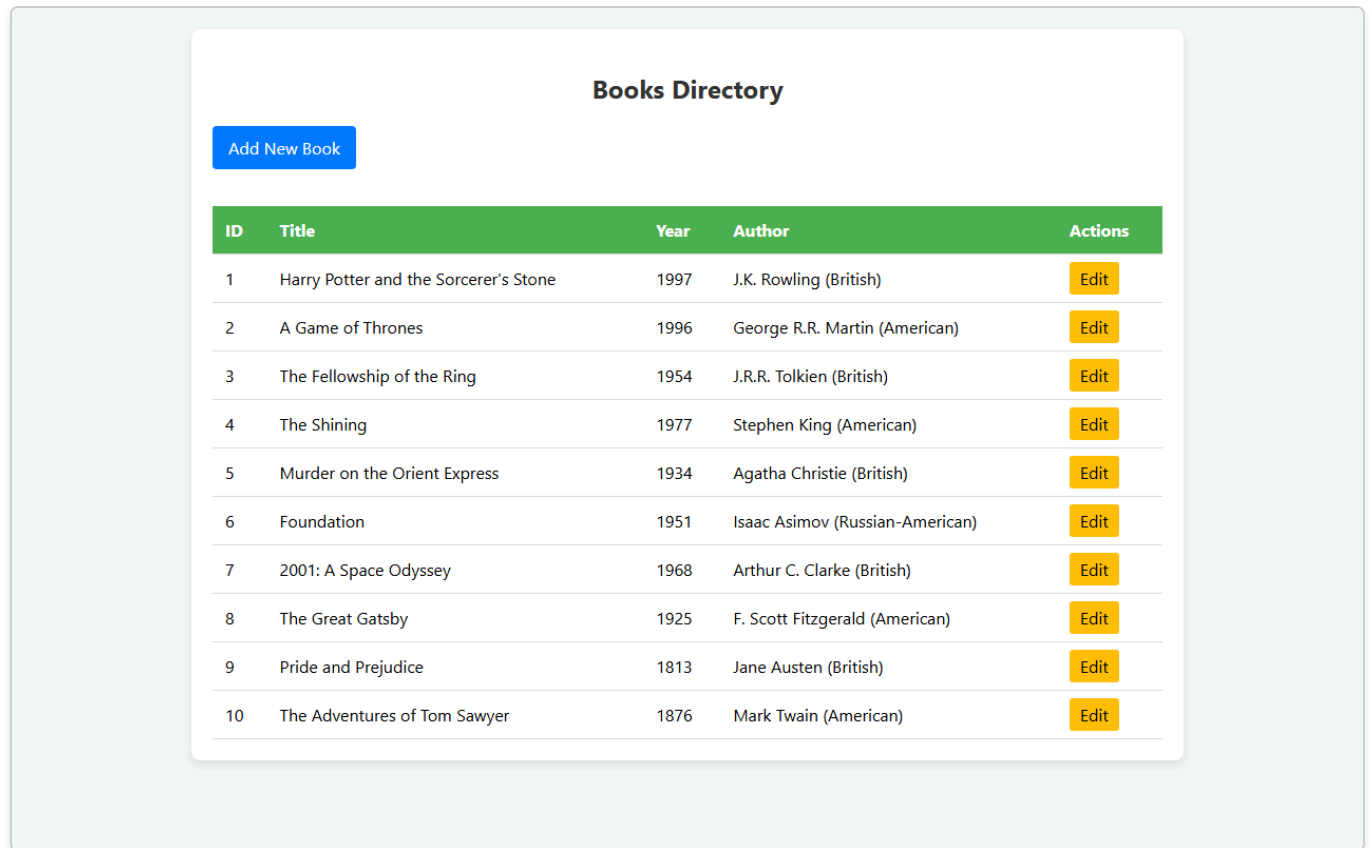
[Save Book](#)

[Back to Book List](#)

Read Operation (Custom Outer/Inner Join Map)

- **Code implementation:** We utilized the `BookRepository` interface by adding: `java @Query("SELECT b FROM Book b JOIN FETCH b.author") List<Book> findAllBooksWithAuthors();`
- This optimizes data fetching (resolves N+1 issues) by inner-joining the `Author` mapping explicitly. Data is then fed to `book-list.jsp` using standard `c:forEach` tags in JSTL.

Screenshot of Read Operation Interface:

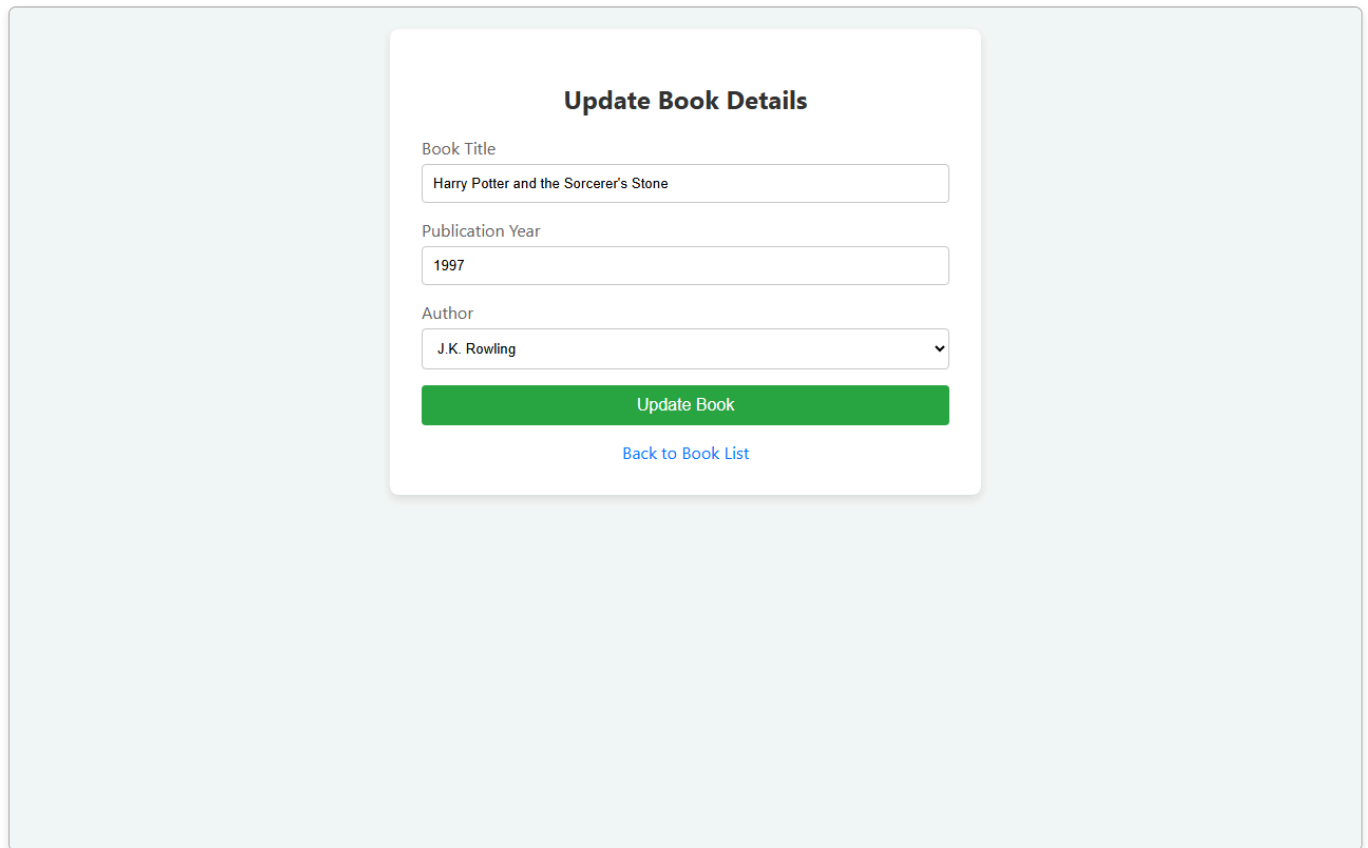


Update Operation

- **Code implementation:** Realized via `showUpdateForm()` mapping and `updateBook()` .
- Fetches the existing Book by ID, and if valid, returns the `book-form.jsp` pre-filled.

Changes made map over the retrieved Entity instance, committing via `libraryService.updateBook()` .

Screenshot of Update Operation Interface:



Update Book Details

Book Title
Harry Potter and the Sorcerer's Stone

Publication Year
1997

Author
J.K. Rowling

Update Book

[Back to Book List](#)

4. Challenges Faced and How We Overcame Them

- **Challenge 1:** *JSP View Resolution with Spring Boot 3+.* Modern Spring Boot leans toward Thymeleaf over JSP. **Solution:** We mapped Tomcat embed Jasper (`tomcat-embed-jasper`) and explicit JSTL dependencies inside the `pom.xml` while ensuring correct `spring.mvc.view.prefix` definitions.
- **Challenge 2:** *N+1 Query Issue when listing Books.* Iterating over Books and accessing `book.author.name` would cause multiple supplementary queries in the JSP layer. **Solution:** Used the requested `JOIN FETCH b.author` inner-join logic inside `BookRepository` to eagerly load dependent author rows in a single query.
- **Challenge 3:** *data.sql running prematurely.* Spring Boot 2.5+ runs data initialization before JPA DDL by default. **Solution:** Updated `application.properties` with `spring.jpa.defer-datasource-initialization=true` resolving any Table-Not-Found exceptions during boot up.

5. Github URL

The project has been committed to the existing Repository path: <https://github.com/dhairya-motta/springboot-data-management>